

Ejemplo relación uno a muchos

Generemos un ejemplo con dos clases codificadas en python donde haya una relación de multiplicidad de uno a muchos de modo de tener que usar un atributo que haga uso de una colección. Veamos la aplicación paso a paso.

Vamos a desarrollar un ejemplo sencillo en Python con las clases **Curso** y **Alumno**.

La relación será:

Un **Curso** puede tener muchos **Alumno**.

Por lo tanto, la clase **Curso** tendrá un atributo de tipo colección llamado **alumnos**, implementado mediante una lista.

1. Relación entre las clases

La multiplicidad será:

Curso 1 — 0..* Alumno

Esto significa:

- Un objeto **Curso** administra una colección de alumnos.
- Un curso puede comenzar sin alumnos.
- Luego se pueden agregar uno o varios alumnos.
- Cada elemento de la colección será un objeto de la clase **Alumno**.

La lista estará declarada dentro de la clase **Curso**:

```
<> Python  
self.alumnos = []
```

2. Estructura del proyecto

Podemos organizar la aplicación en tres archivos:

```
curso_alumnos/  
|  
├─ alumno.py  
├─ curso.py  
└─ app.py
```

Cada clase se guardará en su propio archivo y `app.py` será el punto de entrada de la aplicación.

3. Clase Alumno

Creamos el archivo:

```
alumno.py
```

Código completo

</> Python

```
class Alumno:  
  
    def __init__(self, nombre, apellido, documento):  
        self.nombre = nombre  
        self.apellido = apellido  
        self.documento = documento  
  
    def mostrar_datos(self):  
        return (  
            f"Nombre: {self.nombre} {self.apellido} | "  
            f"Documento: {self.documento}"  
        )  
  
    def __str__(self):  
        return f"{self.nombre} {self.apellido} - DNI: {self.documento}"
```

Explicación

El constructor recibe los datos necesarios para crear un alumno:

</> Python

```
def __init__(self, nombre, apellido, documento):
```

Luego los guarda como atributos del objeto:

</> Python

```
self.nombre = nombre
self.apellido = apellido
self.documento = documento
```

El método:

</> Python

```
def mostrar_datos(self):
```

devuelve una cadena con la información del alumno.

El método especial:

</> Python

```
def __str__(self):
```

determina cómo se mostrará el objeto cuando se use directamente con `print()`.

Por ejemplo:

</> Python

```
alumno = Alumno("Ana", "Gómez", "40111222")
print(alumno)
```

Mostrará:

```
Ana Gómez - DNI: 40111222
```

4. Clase Curso

Creamos el archivo:

```
curso.py
```

Código completo

</> Python

```
class Curso:

    def __init__(self, nombre, codigo):
        self.nombre = nombre
        self.codigo = codigo
        self.alumnos = []
```

```
def agregar_alumno(self, alumno):
    self.alumnos.append(alumno)
    print(f"Alumno agregado: {alumno.nombre} {alumno.apellido}")

def mostrar_alumnos(self):
    print(f"\nAlumnos del curso {self.nombre}:")

    if len(self.alumnos) == 0:
        print("El curso todavía no tiene alumnos.")
    else:
        for alumno in self.alumnos:
            print(alumno)

def cantidad_alumnos(self):
    return len(self.alumnos)
```

5. El atributo que representa la multiplicidad

La parte más importante del ejemplo es:

```
</> Python
self.alumnos = []
```

Este atributo es una lista.

La lista permite que un solo objeto `Curso` tenga asociados muchos objetos `Alumno`.

Al crear el curso:

```
</> Python
curso = Curso("Programación en Python", "PY01")
```

la lista comienza vacía:

```
</> Python
[]
```

Después de agregar alumnos, podría contener:

```
</> Python
[
    alumno1,
    alumno2,
```

```
    alumno3  
]
```

La relación uno a muchos no se representa mediante varios atributos individuales:

```
<> Python  
  
self.alumno1  
self.alumno2  
self.alumno3
```

Se representa mediante una colección:

```
<> Python  
  
self.alumnos
```

Esto permite agregar una cantidad variable de objetos.

6. Agregar objetos a la colección

El método encargado de agregar alumnos es:

```
<> Python  
  
def agregar_alumno(self, alumno):  
    self.alumnos.append(alumno)
```

El parámetro `alumno` recibe un objeto de la clase `Alumno`.

El método `append()` incorpora ese objeto al final de la lista:

```
<> Python  
  
self.alumnos.append(alumno)
```

Por ejemplo:

```
<> Python  
  
alumno1 = Alumno("Ana", "Gómez", "40111222")  
  
curso.agregar_alumno(alumno1)
```

Después de ejecutar esa instrucción, la lista del curso será equivalente a:

```
<> Python  
  
[alumno1]
```

Si agregamos otro:

</> Python

```
alumno2 = Alumno("Carlos", "Pérez", "38999888")

curso.agregar_alumno(alumno2)
```

La colección tendrá dos objetos:

</> Python

```
[alumno1, alumno2]
```

7. Aplicación principal

Creamos el archivo:

app.py

Código completo

</> Python

```
from alumno import Alumno
from curso import Curso

def main():

    print("=== SISTEMA DE GESTIÓN DE CURSOS ===")

    # Creación de un objeto de la clase Curso
    curso_python = Curso(
        "Programación Orientada a Objetos con Python",
        "P00-PY-01"
    )

    # Creación de objetos de la clase Alumno
    alumno1 = Alumno(
        "Ana",
        "Gómez",
        "40111222"
    )

    alumno2 = Alumno(
        "Carlos",
        "Pérez",
```

```
        "38999888"
    )

    alumno3 = Alumno(
        "Lucía",
        "Fernández",
        "42555666"
    )

    # Se muestran Los alumnos antes de agregarlos
    curso_python.mostrar_alumnos()

    # Se agregan Los objetos Alumno a La colección del Curso
    curso_python.agregar_alumno(alumno1)
    curso_python.agregar_alumno(alumno2)
    curso_python.agregar_alumno(alumno3)

    # Se muestran Los alumnos almacenados
    curso_python.mostrar_alumnos()

    # Se consulta La cantidad de alumnos
    cantidad = curso_python.cantidad_alumnos()

    print(f"\nCantidad total de alumnos: {cantidad}")

if __name__ == "__main__":
    main()
```

8. Paso a paso de la ejecución

Paso 1: Importar las clases

</> Python

```
from alumno import Alumno
from curso import Curso
```

La aplicación necesita importar las clases definidas en los otros archivos.

Paso 2: Crear el curso

</> Python

```
curso_python = Curso(
    "Programación Orientada a Objetos con Python",
```

```
"POO-PY-01"  
)
```

Se crea un objeto de la clase `Curso`.

Internamente tendrá los siguientes valores:

```
</> Python  
curso_python.nombre
```

```
Programación Orientada a Objetos con Python
```

```
</> Python  
curso_python.codigo
```

```
POO-PY-01
```

Y la colección comenzará vacía:

```
</> Python  
curso_python.alumnos
```

```
[]
```

Paso 3: Crear los alumnos

```
</> Python  
alumno1 = Alumno("Ana", "Gómez", "40111222")  
alumno2 = Alumno("Carlos", "Pérez", "38999888")  
alumno3 = Alumno("Lucía", "Fernández", "42555666")
```

En este momento existen tres objetos de la clase `Alumno`, pero todavía no pertenecen al curso.

La lista continúa vacía:

```
</> Python  
[]
```


Paso 4: Mostrar el curso antes de agregar alumnos

```
</> Python
curso_python.mostrar_alumnos()
```

Como la lista está vacía, se ejecuta esta condición:

```
</> Python
if len(self.alumnos) == 0:
```

La salida será:

```
Alumnos del curso Programación Orientada a Objetos con Python:
El curso todavía no tiene alumnos.
```

Paso 5: Agregar los alumnos

```
</> Python
curso_python.agregar_alumno(alumno1)
curso_python.agregar_alumno(alumno2)
curso_python.agregar_alumno(alumno3)
```

Cada llamada envía un mensaje al objeto `curso_python`.

Por ejemplo:

```
</> Python
curso_python.agregar_alumno(alumno1)
```

Puede interpretarse como:

Objeto `curso_python`, agregá al objeto `alumno1` a tu colección.

Después de las tres operaciones, la lista contiene:

```
</> Python
[
    alumno1,
    alumno2,
    alumno3
]
```

Paso 6: Recorrer la colección

La colección se recorre dentro del método:

</> Python

```
def mostrar_alumnos(self):
```

Mediante un ciclo `for`:

</> Python

```
for alumno in self.alumnos:  
    print(alumno)
```

En cada iteración, la variable `alumno` hace referencia a uno de los objetos almacenados.

La ejecución conceptual sería:

Primera vuelta: `alumno = alumno1`

Segunda vuelta: `alumno = alumno2`

Tercera vuelta: `alumno = alumno3`

Paso 7: Obtener la cantidad de alumnos

</> Python

```
cantidad = curso_python.cantidad_alumnos()
```

El método utiliza:

</> Python

```
len(self.alumnos)
```

para obtener la cantidad de elementos almacenados en la lista.

Como agregamos tres objetos, devolverá:

3

9. Salida esperada

Al ejecutar la aplicación, veremos algo similar a:

=== SISTEMA DE GESTIÓN DE CURSOS ===

Alumnos del curso Programación Orientada a Objetos con Python:

El curso todavía no tiene alumnos.

Alumno agregado: Ana Gómez

Alumno agregado: Carlos Pérez

Alumno agregado: Lucía Fernández

Alumnos del curso Programación Orientada a Objetos con Python:

Ana Gómez - DNI: 40111222

Carlos Pérez - DNI: 38999888

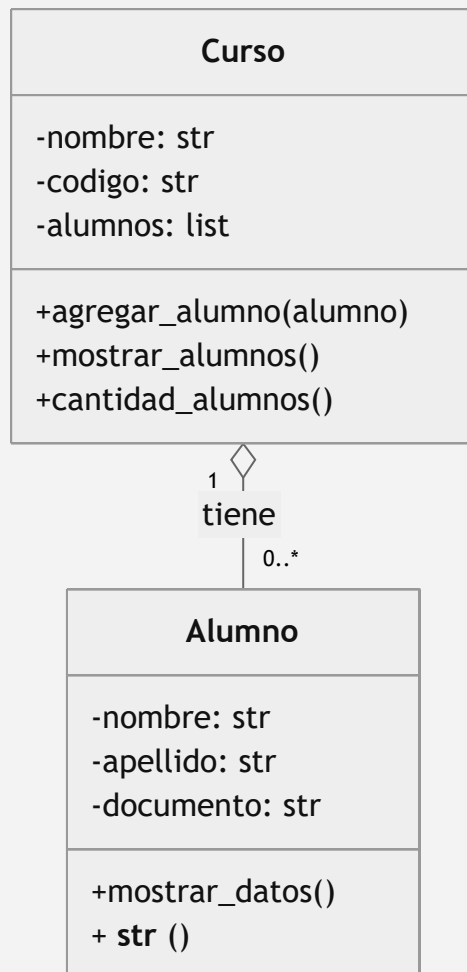
Lucía Fernández - DNI: 42555666

Cantidad total de alumnos: 3

10. Diagrama UML

Podemos representar la relación mediante Mermaid:

 Mermaid



El símbolo de rombo vacío representa una relación de **agregación**.

Esto resulta apropiado porque:

- El curso contiene una colección de alumnos.
- Los alumnos pueden existir independientemente del curso.
- Un alumno se crea antes de ser agregado al curso.
- Eliminar el curso no implica necesariamente eliminar los objetos `Alumno`.

11. Cómo ejecutar el proyecto desde VS Code

Abrimos la carpeta del proyecto en Visual Studio Code:

curso_alumnos

Verificamos que estén los tres archivos:

```
alumno.py  
curso.py  
app.py
```

Luego abrimos una terminal desde:

```
Terminal → Nueva terminal
```

Ejecutamos:

```
</> Bash  
python app.py
```

En algunos equipos Windows puede ser necesario utilizar:

```
</> Bash  
py app.py
```

La ejecución siempre comienza en:

```
</> Python  
if __name__ == "__main__":  
    main()
```

12. Versión completa en un único archivo

Para una primera explicación en clase, también podemos colocar todo en un solo archivo:

```
</> Python  
class Alumno:  
  
    def __init__(self, nombre, apellido, documento):  
        self.nombre = nombre  
        self.apellido = apellido  
        self.documento = documento  
  
    def mostrar_datos(self):  
        return (  
            f"Nombre: {self.nombre} {self.apellido} | "  
            f"Documento: {self.documento}"  
        )  
  
    def __str__(self):  
        return f"{self.nombre} {self.apellido} - DNI: {self.documento}"
```

```
class Curso:

    def __init__(self, nombre, codigo):
        self.nombre = nombre
        self.codigo = codigo
        self.alumnos = []

    def agregar_alumno(self, alumno):
        self.alumnos.append(alumno)
        print(f"Alumno agregado: {alumno.nombre} {alumno.apellido}")

    def mostrar_alumnos(self):
        print(f"\nAlumnos del curso {self.nombre}:")

        if len(self.alumnos) == 0:
            print("El curso todavía no tiene alumnos.")
        else:
            for alumno in self.alumnos:
                print(alumno)

    def cantidad_alumnos(self):
        return len(self.alumnos)

def main():

    print("=== SISTEMA DE GESTIÓN DE CURSOS ===")

    curso_python = Curso(
        "Programación Orientada a Objetos con Python",
        "P00-PY-01"
    )

    alumno1 = Alumno(
        "Ana",
        "Gómez",
        "40111222"
    )

    alumno2 = Alumno(
        "Carlos",
        "Pérez",
        "38999888"
    )

    alumno3 = Alumno(
        "Lucía",
        "Fernández",
        "42555666"
```

```
)

curso_python.mostrar_alumnos()

curso_python.agregar_alumno(alumno1)
curso_python.agregar_alumno(alumno2)
curso_python.agregar_alumno(alumno3)

curso_python.mostrar_alumnos()

print(
    f"\nCantidad total de alumnos: "
    f"{curso_python.cantidad_alumnos()}"
)

if __name__ == "__main__":
    main()
```

Este ejemplo demuestra la multiplicidad **uno a muchos**, porque un único objeto `Curso` mantiene una lista que puede almacenar muchos objetos `Alumno`.



¿Te ha sido útil esta conversación hasta ahora?